Laptop Shop Assist AI 2.0

Project workflow and steps involved:

Introduction:

- **Background**: The evolution of AI-driven systems has transformed user interactions with technology. The Laptop Shop Assistant AI 2.0 builds upon the existing ShopAssistAI by integrating advanced OpenAI function calling capabilities to provide a more dynamic and personalized shopping experience for users.
- **Requirement**: To demonstrate how integrating OpenAI's Function Calling API enhances the chatbot's ability to perform tasks beyond simple responses, enabling seamless and real-time personalized laptop recommendations based on user preferences.
- **Key Focus**: This presentation will focus on the process of integrating the OpenAI Function Calling API into ShopAssistAI, allowing for advanced interactions such as executing functions, accessing external data, and delivering tailored results based on complex user inputs.
- **Purpose**: Improve the user experience with more interactive and context-aware features, making the assistant a smarter tool for laptop shopping.

Problem Statement:

In ShopAssistAI, while the chatbot provided laptop recommendations, there were issues with maintaining consistency in user interactions and ensuring modularity in the underlying system architecture. As new features were added, the system became more difficult to scale and maintain, leading to inconsistent responses and limited adaptability to new user needs.

Issue:

These challenges led to a suboptimal user experience, with customers encountering occasional discrepancies in recommendations and the inability to seamlessly integrate new features or improve existing ones.

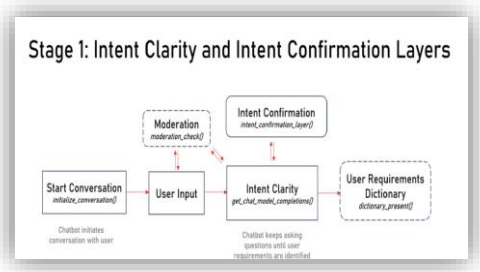
Solution (Laptop- Shop Assist AI 2.0):

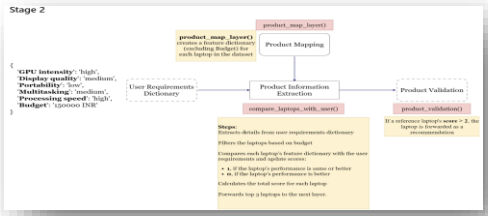
Laptop-Shop-Assist-AI 2.0 addresses these issues by integrating OpenAI's Function Calling API, which allows the chatbot to access dynamic functionalities and handle complex tasks modularly and consistently. This upgrade helps that each function is designed as a modular unit, making the system scalable and easier to maintain, while delivering consistent, accurate, and personalized recommendations using various stages

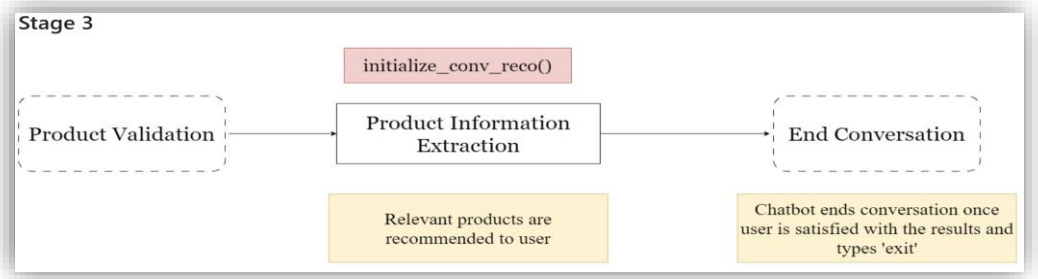
Stage 1: Intent Clarity and Intent Confirmation Layers

Stage 2: Product Mapping

Stage3: Product Validation







Approach:

- Built a Modular Architecture for better scalability with function-calling integration enhance chatbot capabilities and provide personalized laptop recommendations.
- Improved conversation flow for consistent, safe, and efficient user interactions.
- Automatically scans user inputs to detect harmful or unsafe language, ensuring a secure user environment that acts in real-time.
- Developed user-friendly operations using HTML, CSS, and JavaScript for a responsive and intuitive interface.

Improvisations:

Function Calling API Integration:

- Updated get_chat_completions_tool to leverage the Function Calling API for precise handling of user inputs and preferences extraction.
- Added shopassist_custom_functions to define the structure for extracting user preferences regarding laptop attributes.
- Enhanced Moderation checks for both user inputs and assistant responses to maintain appropriate and safe conversations.
- Structured Communication : Structured interaction flow between frontend and backend by using JSON for smooth data exchange.

Simpler Code:

- Modified the current dictionary_present() function to a slight easier version of code from the existing one.

```python
def dictionary_present(response_dict):
    """
    Check if the dictionary is not empty and contains data.

    Parameters:
    response_dict (dict): The dictionary to check.

    Returns:
    bool: True if the dictionary is not empty, False otherwise.
    """
    if isinstance(response_dict, dict) and bool(response_dict):
        return True
    else:
        return False
```

- Made changes in the custom function code. Function, named ‘extract_user_info’, is designed to extract structured information from user input regarding their laptop preferences.

- Updated the code and included variable for easier identification of tagged dictionaries.

```python
# Update these response dictionaries to match the structure of the tagged dictionaries
gamer_response_dict = {'GPU intensity': 'high', 'Display quality': 'high', 'Portability': 'low', 'Multitasking': 'high', 'Processing sp
academic_response_dict = {'GPU intensity': 'high', 'Display quality': 'medium', 'Portability': 'medium', 'Multitasking': 'medium', 'Pro
business_response_dict = {'GPU intensity': 'low', 'Display quality': 'high', 'Portability': 'high', 'Multitasking': 'high', 'Processing

# Now run your existing code:
print(f"Case 1 - Gaming conversation")
gamer_score = find_match_score(gamer_tagged_dict, gamer_response_dict, debug=True)
print(f"Score : {gamer_score}")
print("="*50)

print(f"Case 2 - Academic conversation")
academic_score = find_match_score(academic_tagged_dict, academic_response_dict, debug=True)
print(f"Score : {academic_score}")
print("="*50)

print(f"Case 3 - Business conversation")
business_score = find_match_score(business_tagged_dict, business_response_dict, debug=True)
print(f"Score : {business_score}")
print("="*50)
```

Future Scope :

- Output Format inconsistency: The output format for each layer of the classification process lacks uniformity. To address this, leveraging the function API of the GPT model can help standardize the output format, ensuring consistency with the input request specifications. This will facilitate smoother interactions and more predictable responses.
- Rule Framework: Expanding the rule set to cover a wider variety of criteria and edge cases will provide the LLM with a more comprehensive context, leading to improved classification accuracy and fewer misclassifications.
- Misclassification Issues: Fine-tuning an open-source LLM with a more focused dataset, reflecting the nuanced nature of these classifications, can significantly improve the model's understanding and performance in this domain.
- Intent Recall in Dialogue Flow: The current dialogue flow cannot recall previously extracted products when the user’s intent changes. Introducing an additional layer to detect shifts in intent and flagging it for product extraction recall will improve the adaptability of the solution
- Vector Embedding: This method can significantly improve the relevance matching process, allowing for more precise identification of the most relevant products for the user’s needs in an efficient manner.
- Workflow Templatization: The workflow and solution framework can be templated for chatbot applications across various product domains. This approach will give the LLM the necessary contextual understanding for different industries, expanding its utility beyond just laptops.

Glossary:

- AI (Artificial Intelligence): Machines programmed to simulate human intelligence, think, and
- learn.
- API (Application Programming Interface): A set of rules for software communication.

- Chatbot: Software for conducting online text or speech-based conversations.
- OpenAI: A research organization and company developing AI technologies, including the GPT models
- Pandas: A Python library for data manipulation and analysis.
- GPT (Generative Pre-trained Transformer): A deep learning model by OpenAI that generates human-like text.
- HTML (HyperText Markup Language): Standard language for web page structure.
- CSS (Cascading Style Sheets): Language for styling HTML and XML documents.
- JavaScript: Programming language for creating interactive web elements

References:

The project references course materials are taken from upGrad's curriculum.

- The project references presentations in upGrad's recorded module/live sessions given by Kshitij Jain, Kaustubh Singh, Sheshanth AS.